

COMPILER CONSTRUCTION SEMESTER PROJECT

NexaLex Compiler Studio

A Multi-Language Lexical Analyzer for C, C++, Java, JavaScript, C#, and Python

TOKENIZATION | SYMBOL TABLE | ERROR RECOVERY | VISUAL ANALYTICS

Submitted By: _____

Registration No.: _____

Submitted To: _____

Department: Computer Science / Software Engineering

Technology: Python 3, Tkinter, ttk, Object-Oriented Design

Academic Session: 2025-2026

PROJECT REPORT

1. Introduction and Project Scope

A compiler translates source code through a sequence of phases. Lexical analysis is the first phase: it groups characters into lexemes and emits tokens for later phases. NexaLex implements profile-driven analysis for C, C++, Java, JavaScript, C#, and Python through an interactive desktop workbench.

The application is both a working analyzer and a teaching aid. It exposes the token stream, inferred symbols, diagnostics, and aggregate statistics in separate views. Valid and invalid samples support a clear semester-project demonstration.

1.1 Objectives

Objective	Implementation Outcome
Token recognition	Recognize keywords, identifiers, numeric literals, strings, characters, operators, and separators.
Comment handling	Consume single-line and multi-line comments without adding them to the token stream.
Symbol management	Record identifier, inferred type, scope, and declaration line.
Error detection	Report malformed identifiers, invalid characters, unknown tokens, and unterminated constructs.
Multi-language	Select or automatically detect six supported programming languages.
Presentation	Provide Fluent navigation, search, highlighting, charts, samples, and CSV export.

1.2 Supported Languages

Language	Extensions	Profile Highlights
C / C++	.c, .h, .cpp, .hpp	Preprocessor tokens, C comments, types, classes, and scope operators.
Java / C#	.java, .cs	Object-oriented keywords, built-in types, modifiers, and nullable operators.
JavaScript	.js, .jsx, .mjs	Dynamic declarations, arrow operators, and template strings.
Python	.py, .pyw	Hash comments, triple strings, Python keywords, and operators.

2. System Design and Architecture

The project follows a modular object-oriented architecture. Each module owns one major responsibility, preventing scanning rules from becoming entangled with interface code. The GUI requests an analysis, receives one AnalysisResult object, and renders its tokens, symbols, errors, and statistics.

2.1 Module Responsibilities

Module	Responsibility	Primary Classes
main.py	Starts the application and Tk event loop.	main()
languages.py	Profiles, extensions, keywords, comments, operators, and detection.	LanguageProfile
lexer.py	Profile-driven scanning, classification, statistics, and recovery.	Lexer, AnalysisResult
token.py	Represents tokens while preserving standard-module compatibility.	Token
symbol_table.py	Stores unique symbols by name and scope.	Symbol, SymbolTable
errors.py	Represents lexical diagnostics.	LexicalError
gui.py	Fluent workbench, sidebar, samples, themes, charts, and exports.	LexicalAnalyzerGUI

2.2 Processing Flow

Step 1 - Input: Enter source, open a file, or load a bundled sample.

Step 2 - Detect: Auto Detect uses extension and lightweight source hints.

Step 3 - Scan: The Lexer applies the selected profile with longest-match lookahead.

Step 4 - Present: The workbench populates tokens, symbols, diagnostics, charts, and status.

2.3 Scanner State

The scanner maintains the source, current index, line, column, token and error collections, symbol table, scope stack, pending scope, and declaration context. This makes the scan deterministic and gives every output a useful source position.

3. Implementation, Interface, and Testing

3.1 Lexical Algorithm

At each position the lexer selects a specialized routine from the current character. Whitespace is consumed; letters and underscores begin identifiers; digits begin numbers; quotes begin literals; slash with lookahead begins a comment; and remaining characters are checked against operators and separators. Multi-character operators are checked first to enforce the longest-match rule.

3.2 Professional GUI Features

Feature	Implementation
Fluent layout	Icon sidebar, glass-style command header, and page workbench.
Rounded surfaces	Canvas cards and buttons with soft layered shadows.
Typography	Inter when installed, with Segoe UI fallback.
Interaction	Smooth hover transitions on navigation and actions.
Analytics	Rounded token bars and radial scan-quality visualization.
Productivity	Auto Detect, language selector, samples, search, shortcuts, and exports.

3.3 Test Summary

Test Case	Expected Result	Status
Six valid samples	Every language auto-detected with zero lexical errors.	Passed
Invalid identifier	Malformed name reported without stopping.	Passed
Literal errors	Unterminated string and invalid character reported.	Passed
Comment error	Missing closing delimiter detected.	Passed
Unknown input	Unsupported symbols reported with line data.	Passed
GUI integration	Navigation, samples, themes, and charts initialize.	Passed

4. Results, Limitations, and Conclusion

4.1 Demonstrated Results

All six valid samples are detected from their extensions and produce zero lexical errors. The invalid set demonstrates recovery across malformed identifiers, invalid characters, unknown operators, unterminated strings, and unterminated comments. A JSON manifest stores reproducible statistics for every sample.

Evaluation Area	Observed Result
Language coverage	C, C++, Java, JavaScript, C#, and Python use dedicated profiles.
Correctness	Common keywords, operators, separators, literals, comments, and identifiers are handled.
Traceability	Tokens, symbols, and errors include source line information.
Usability	Sidebar navigation, detection, samples, themes, charts, shortcuts, and exports support demonstration.
Maintainability	Scanning, models, symbols, errors, and presentation remain separate.

4.2 Current Limitations

Language subset: Profiles cover common lexical elements rather than every contextual compiler rule.

Numeric formats: Hexadecimal, scientific notation, and suffixes are not implemented.

Semantic depth: Type and scope inference is lightweight rather than parser-authoritative.

4.3 Future Enhancements

Parser integration: Build an abstract syntax tree and report syntax errors.

Automata visualization: Display state transitions for selected token patterns.

Configuration: Load language keywords and operators from a definition file.

Testing: Add a formal automated suite and coverage report.

4.4 Conclusion

NexaLex demonstrates profile-driven lexical analysis in a modern desktop environment. Its six-language scanner, automatic detection, sample library, modular design, recovery, symbol tracking, Fluent-inspired interface, animated controls, dark mode, and visual statistics make it suitable for academic evaluation and live demonstration.